

Spring WebFlux + ChatGPT 실전 구현

목차

0 장 - 강의 소개

1 장 - WebFlux, 어디서부터 시작할까?

- 1.1 리액티브 프로그래밍이란?
- 1.2 스레드란?
- 1.3 블로킹과 비동기 프로그래밍이란?
- 1.4 Reactive Stream – 함수형 프로그래밍이란?
- 1.5 WebFlux 의 Subscriber – Publisher 패턴
- 1.6 Netty 와 이벤트 루프 패턴
- 1.7 WebFlux 는 Netty 와 어떻게 통합 될까?
- 1.8 가장 중요 – WebFlux 에서는 블로킹 처리를 어떻게 해야 할까?

2 장 - WebFlux, 어떻게 구현할까?

- 2.1 WebFlux 의 기본적인 사용법, Flux 와 Mono
- 2.2 Operator 에 대하여 - Mono
- 2.3 Operator 에 대하여 - Flux
- 2.4 Operator FlatMap 에 대하여
- 2.5 시그널과 예외처리에 대하여
- 2.6 Scheduler 에 대하여

3 장 - WebFlux + LLM 심화 구현

- 3.1 GPT Stream 통신 구현 - WebClient, GPT API 연동
- 3.2 GPT Stream 통신 구현 - React 프로젝트 연동
- 3.3 GPT Stream 통신 구현 - stream 통신에 대하여
- 3.4 GPT Stream 통신 구현 - 실전 예외 처리 구현
- 3.6 Chain Of Thought 을 이용한 GPT 생각 전개 기능

0 장 강의 소개

본 강의 자료는 최소한의 텍스트만 포함하고 있습니다.

자세한 사항은 강의에서 확인해주세요.

Spring WebFlux 란?

- 저희가 WebFlux 를 사용하고 싶다는건 리액티브한 서비스를 만들고 싶다는 의미라고 보셔도 됩니다.
- WebFlux 는 **리액티브한 서비스**를 쉽게 개발할 수 있도록 도와주는 스프링 프레임워크의 라이브러리입니다.
- 리액티브 서비스에서는 비동기 웹 서버의 활용이 중요한데, WebFlux 는 Netty 와 같은 비동기 웹 서버와 스프링 애플리케이션을 **자연스럽게 통합**할 수 있도록 지원합니다.
- 이 외에도 스트림 방식의 데이터 처리, 논 블로킹 API 제공 등 다양한 리액티브 기능들을 포함하고 있습니다.

리액티브한 서비스란?

- "리액티브하다"는 것은 **요청에 빠르고 효율적으로 반응**한다는 의미입니다.
- 사용자의 요청이 들어왔을 때 **자원을 효율적으로 활용하여 신속하게 처리하고, 즉각 즉각 반응하는 서비스**라고 할 수 있을것 같습니다.
- 이렇게 "**리액티브**"하게 만들기 위해서는 **비동기 웹 서버**의 활용이 필수적입니다.

왜 리액티브한 서비스에 비동기 웹 서버가 필수일까?

여기 치킨을 시키고 기다리는 두 사람이 있습니다.

두 사람은 각각 Tomcat 이라는 방과 Netty 라는 방에 들어가 있습니다.

동기 블로킹 방식 (Tomcat 이라는 방의 사람)

- 이 사람은 치킨이 도착할 때까지 **아무것도 하지 않고 문 앞에서 기다립니다.**
왜냐면 컴퓨터 세계에서는 벨 같은게 없어서 치킨을 받을라면 문 앞에서 계속

감시하고 있는 방법 밖에는 없기 때문입니다.

이 사람은 치킨을 시켰으면 치킨이 올 때까지 문 앞에서 기다리고

치킨과 동기적(Synchronous)으로 동작합니다.

여기서 아무것도 하지 않고 방문 앞에서 기다리고 있는 상황을 블로킹이라고 합니다.

(치킨이 올 때까지 블로킹)

비동기 논블로킹 방식 (Netty 이라는 방의 사람)

- 이 사람은 치킨이 도착할 때까지 TV도 보고, 문자도 보냅니다.

이 사람은 치킨을 시킨 후에 치킨이 오는걸 기다리지 않고

치킨과 비동기적(Asynchronous)으로 동작합니다.

대신 엄마(OS)에게 부탁해서 문 앞에서 치킨이 도착하는지 확인하고, 도착하면

거실(커널 영역)에 메모를 해달라고 합니다.

설거지가 쌓였거나, 핸드폰 알람이나 다른 어떤 이벤트가 발생하더라도 거실에 메모를 해달라고 부탁합니다.

이 사람은 작업을 하나 완료할 때 마다 거실로 가서 메모를 확인(Selector)합니다.

메모만 확인하면 되기 때문에 이 사람은 절대로 대기하지 않습니다.

(치킨이 오기 전까지 논블로킹하게 여러 작업 처리)

누가봐도 문 앞에서 멍하니 기다리고 있는 Tomcat 의 사람보다

대기를 하지 않고 계속 작업하는 Netty 의 사람이 "반응을 잘하는 사람", 즉 리액티브한 사람입니다.

방 = 프로세스

사람 = 스레드

치킨 시키기 = I/O 즉, 외부 리소스가 필요한 작업

치킨 도착 = 이벤트

엄마 = OS (I/O 다중화 메커니즘)

거실 = 커널 영역

거실에서 메모 확인 = 이벤트 루프 Selector 로 이벤트 조회

치킨을 시키는걸 앞서 보여드렸던 GPT API 호출에 빗낼 수 있을것 같습니다.

리액티브한 서비스의 스레드는 이런 이벤트(I/O)에 효율적으로 반응하며 대기 없이 작업을

처리 할 수 있어야 하기 때문에

비동기 논블로킹 방식을 사용하는 웹 서버를 사용하는건 필수적입니다.

그렇다면 WebFlux 는 리액티브한 서비스에 왜 필요할까?

비동기 웹 서버는 우리가 직접 구현하는게 아니고 이미 잘 만들어진 것을 가져다 사용하면 됩니다. 따라서 개념만 잘 이해하고 있다면 문제가 없습니다.

그렇다면 우리가 원하는 리액티브한 코드를 비동기 웹 서버 위에서 어떻게 구현할 수 있을까요?

바로 그 역할을 해주는것이 Spring WebFlux 입니다.

- 기존 스프링이 Tomcat/Servlet 과 완벽히 통합되는 것처럼, WebFlux 는 **Netty 와의 완벽한 통합**을 지원합니다.
- **WebClient, R2DBC** 등 다양한 비동기 API 와 쉽게 연동 가능
- **Publisher-Subscriber 패턴(리액티브 스트림즈 구현체 - Flux Mono)**을 기반으로 **Netty 의 이벤트 루프 패턴과 자연스럽게 통합**
- 스레드 변경이 필요한 경우에는 **Scheduler** 를 통해 유연하게 병렬/비동기 처리가 가능
- Stream 통신을 통해 결과물이 **부분적으로 완성될 때마다 사용자에게 즉시 전달** 가능
- 등등..

1 장 WebFlux, 어디서부터 시작할까?

1.1 리액티브 프로그래밍이란?

이번 장에서는 **WebFlux** 를 제대로 사용하기 위해 반드시 알아야 할 개념들을 살펴보겠습니다.

"리액티브 프로그래밍, 비동기, 함수형, 스레드, 블로킹, 이벤트 루프, 비동기 웹 서버..."

처음 들으면 다 따로 노는 이야기처럼 들릴 수 있지만, WebFlux 에서는 이 모든 개념이 서로 긴밀하게 연결되어 있습니다.

이 개념들을 따로따로 공부하면 어렵게 느껴질 수 있지만, **WebFlux** 라는 하나의 흐름 안에서 보면 자연스럽게 이해할 수 있습니다.

이 강의에서는 바로 그 흐름을 잡아드리겠습니다.

실습 준비: 개발 환경 구성

WebFlux 실습을 위해 필요한 기본 툴을 먼저 준비하겠습니다.

- **IntelliJ IDEA**
 - 자바 개발에 특화된 강력한 IDE 입니다.
 - 무료 체험판(30 일)을 사용할 수 있으므로, 아직 사용하지 않으신 분들은 설치를 권장드립니다.
 - 기존에 Eclipse 를 사용하고 계신 분은 그대로 진행하셔도 괜찮습니다.
- **실습용 프로젝트**
 - 직접 프로젝트를 생성하셔도 되고, 강의 자료에 포함된 zip 파일을 통해 바로 실습하셔도 됩니다.

- GitHub 링크를 통해 각 강의 실습 코드를 제공해드릴 예정입니다.
 - **Postman**
 - HTTP 요청을 테스트하는 도구로, API 테스트에 매우 유용합니다.
-

리액티브 프로그래밍을 코드로 이해하기

WebFlux의 핵심인 **리액티브 프로그래밍**을 예제로 직접 경험해보겠습니다.

전통적인 방식

1 부터 9 까지의 숫자를 리스트에 담고, 클라이언트에 반환하는 단순한 API를 먼저 만들어 봅니다.

```
List<Integer> list = new ArrayList<>();
for (int i = 1; i <= 9; i++) {
    Thread.sleep(500); // 0.5 초 지연
    list.add(i);
}
return list;
```

이 방식은 데이터를 다 준비한 후에 한 번에 클라이언트로 응답을 보내기 때문에 **총 5 초가 소요**됩니다.

이때 사용된 `Thread.sleep()`은 스레드를 멈추는 블로킹 코드입니다.

리액티브 방식으로 바꾸기

리스트 대신 **Flux**를 사용하면, 데이터를 준비되는 대로 즉시 클라이언트에 전송할 수 있습니다.

```
Flux.create(sink -> {
    for (int i = 1; i <= 9; i++) {
        Thread.sleep(500); // 실제로는 블로킹이지만 예시로 사용
```

```
        sink.next(i);
    }
    sink.complete();
});
```

이제 클라이언트는 0.5 초마다 하나씩 응답을 받아보게 됩니다.

이를 가능하게 하는 것이 바로 Flux 이고, 브라우저에서는 http/event-stream 방식으로 이 스트림을 수신하게 됩니다.

리액티브 프로그래밍의 핵심 포인트

1. 리액티브 스트림 사용 (Flux / Mono)

- 데이터를 하나씩 스트리밍처럼 방출하며, 각 요소를 준비되는 즉시 전송 가능
- http/event-stream 을 활용하여 실시간으로 클라이언트에 반응

2. 논블로킹으로 작성

- 리액티브의 가장 중요한 특징은 **논블로킹**입니다.
- 우리가 사용하는 서버(Netty)는 비동기 I/O 를 지원합니다.
- 하지만 Thread.sleep() 같은 **블로킹 코드를 작성하면 논블로킹 환경도 블로킹이 됩니다.**
- 따라서 논블로킹을 유지하기 위해 비동기적인 흐름으로 코드를 작성해야 합니다.

Netty 와 WebFlux

Spring WebFlux 는 내부적으로 Netty 라는 비동기 웹서버를 사용합니다.
Netty 는 자체적으로 논블로킹 방식으로 작동하지만, 우리가 작성하는 코드까지 논블로킹해야 전체 시스템이 리액티브하게 동작할 수 있습니다.

예를 들어, 코드에 `Thread.sleep()`이 있으면 아무리 Netty 가 비동기라 해도 블로킹이 발생합니다.

이 점을 이해하면, 왜 우리는 리액티브하게 작성하고, **블로킹을 피해야 하는지** 자연스럽게 알 수 있습니다.

이번 시간의 핵심 요약

1. 리액티브 스트림(Flux, Mono)을 사용해 데이터가 준비되는 대로 전송
2. **논블로킹**하게 코드를 작성해야 리액티브 시스템이 제대로 작동
3. WebFlux 는 단순히 다른 웹 프레임워크가 아닌 **비동기 + 논블로킹 + 함수형 프로그래밍 패러다임**의 조합

다음 시간에는 우리 코드를 실행시켜주고 블로킹이 발생하는 주체인 **스레드**가 무엇인지 살펴보겠습니다.

1.2 스레드란?

우리가 짠 코드를 실행해주는 주체, 바로 스레드(Thread)입니다.

영어 단어 Thread 는 '실'이라는 뜻이죠.

하나로 이어진 작업 흐름을 따라 실행된다는 의미에서, 우리는 **작업 흐름의 실행 단위**를 스레드라고 부릅니다.

즉, 사용자의 요청이 웹 서버에 도달하고, 우리가 작성한 코드가 실행되어 다시 사용자에게 응답이 전달되기까지의 일련의 흐름은 **스레드가 CPU 자원을 얻어가며 처리하는 실행 흐름**인 것입니다.

스레드는 프로세스 안에서 만들어지며, 그 프로세스가 처리해야 하는 모든 작업은 결국 이 스레드를 통해 실행됩니다.

스레드의 역할: 톰캣 vs Netty

톰캣의 스레드 모델

톰캣은 **하나의 요청마다 하나의 스레드**를 배정하는 구조입니다.

- 사용자의 요청이 들어오면,
- 그 요청을 처리할 스레드를 하나 가져오고,
- 요청 수락 → 요청 읽기 → 비즈니스 로직 처리 → 응답 쓰기까지
- 모든 단계가 **하나의 스레드에서 차례대로** 수행됩니다.

이 흐름은 동기적이며, 중간에 I/O 가 끼면 해당 스레드는 **블로킹** 상태에 빠집니다.

하지만 괜찮습니다.

톰캣은 수백 개의 스레드를 미리 만들어 놓고 운용하기 때문에, 몇몇 스레드가 블로킹되어도 남은 스레드로 다른 요청을 처리할 수 있기 때문입니다.

톰캣은 어느 정도의 블로킹을 감당할 수 있는 구조입니다.

Netty 의 스레드 모델

Netty 는 다른 접근을 합니다.

- 스레드 수를 매우 소수로 유지하고,
- 요청이 들어오면 실제 I/O 대기 시간은 OS 에 위임합니다.
- 스레드는 단지 요청을 받고 → 이벤트를 등록하고 → 비즈니스 로직만 짧게 실행하는 방식으로,
- 아주 짧은 시간 동안만 작업을 처리합니다.

예를 들어, 요청이 100 개 들어와도

Netty 는 모든 요청을 한두 개의 스레드만으로 처리할 수 있습니다.

왜냐면 각 요청에서 스레드가 직접 일하는 시간은 0.0001 초도 되지 않기 때문입니다.

이 구조의 핵심은 블로킹이 없다는 가정입니다.

만약 우리가 Thread.sleep() 같은 블로킹 코드를 작성해버리면?

→ Netty 스레드는 4.5 초 동안 아무 일도 못 하고 대기하게 됩니다.

→ 결국 소수 스레드 구조는 깨지고, 서버 전체가 마비될 수 있습니다.

그럼 이런 질문이 생길 수 있습니다.

1. Netty 도 스레드를 100 개 정도 만들어 놓으면 안 되나요?

이론적으로 가능하지만, 스레드를 많이 만든다고 능사는 아닙니다.

스레드가 많아질수록 다음과 같은 문제가 생깁니다.

- CPU의 캐시 히트율 저하
- 컨텍스트 스위칭 비용 증가 (작업 전환 시 스레드 상태를 저장/복원하는 비용)
- 메모리 사용량 증가

Netty는 이 문제를 피하기 위해 물리적인 CPU 코어 수와 같은 수의 스레드만 사용하도록 기본 설정되어 있습니다.

2. CPU 자원은 한정돼 있는데 어떻게 스레드를 마음대로 만들 수 있나요?

사실 우리가 말하는 스레드는 논리적 스레드입니다.

운영체제(OS)는 프로세스들에게 논리 스레드를 필요한 만큼 마음대로 만들고 진짜 스레드라고 가정하고 사용하도록 합니다.

그리고 이 논리 스레드들을 물리적인 CPU 스레드에 할당하여 스케줄링해 실행해줍니다.

즉, 우리는 스레드를 마음대로 만들 수 있지만 실제 실행은 OS가 CPU 코어에 맞게 스케줄링해주는 구조입니다.

정리: 스레드와 서버 아키텍처의 핵심 차이

구분	톰캣	Netty
스레드 모델	요청당 스레드 하나	이벤트 기반, 소수 스레드
블로킹 허용	일정 부분 허용	매우 취약
스레드 수	수백 개	CPU 코어 수와 동일

구분	톰캣	Netty
특징	직관적이지만 자원 소모 많음	고성능, 고효율 구조이지만 논 블로킹 코드 요구

이번 시간의 핵심 요약

1. 스레드란? 하나의 작업 흐름을 실행시켜주는 주체. 우리가 짠 코드도 이 스레드를 통해 실행됩니다.
2. 톰캣은 요청당 하나의 스레드를 사용하며, 블로킹을 어느 정도 감당할 수 있는 구조입니다.
3. Netty 는 소수의 스레드로 대기 없이 고성능 처리를 지향하지만, 블로킹 코드에 매우 취약합니다.
4. 스레드는 논리적 개념이며, 실제 실행은 OS 가 물리 CPU 코어에 맞게 스케줄링합니다.
5. 스레드를 많이 만들면 성능이 떨어질 수 있으므로 Netty 는 효율성을 위해 최소한의 스레드만 사용합니다.

다음 시간에는 스레드가 블로킹되는 상황을 더 깊이 알아보고
이 문제를 해결하기 위한 핵심 전략인 비동기 프로그래밍에 대해
살펴보겠습니다.

1.3 블로킹과 비동기 프로그래밍이란?

블로킹(Blocking) vs 논블로킹(Non-Blocking)

지난 시간에는 스레드의 역할과 서버 구조에 대해 살펴보았습니다.
이번 시간에는 그 스레드가 '블로킹된다'는 게 어떤 의미인지,
그리고 그 블로킹을 어떻게 회피할 수 있을지, 즉 비동기 프로그래밍에 대해
알아보겠습니다.

블로킹이란?

블로킹(Blocking)은 어떤 작업을 처리하던 스레드가 외부 리소스를 기다리느라
멈춘 상태를 말합니다.

작업이 끝날 때까지 스레드는 아무 것도 못 하고 대기하게 되죠.

다음 중 어떻게 블로킹 작업일까요?

작업 예시	블로킹 여부	이유
DB 조회	블로킹	DB 프로세스에서 처리되므로 결과를 기다려야 함
Network I/O (HTTP 통신 등)	블로킹	OS와 네트워크 장비가 처리, 결과를 기다림
파일 읽기/쓰기	블로킹	디스크 장치의 처리 결과를 기다려야 함
객체 생성 / 계산 로직	논블로킹	CPU에서 즉시 처리 가능

요점: 외부 리소스에 의존하는 작업은 대부분 블로킹 작업입니다.

비동기 프로그래밍이란?

비동기 프로그래밍(Asynchronous Programming)이란
작업의 순서를 강제하지 않고, 대기해야 하는 작업은 다른 곳에 맡기고 다음
작업을 계속 진행하는 방식입니다.

예를 들어

- Netty 에서 중요한 스레드(= 이벤트 루프 스레드)는
블로킹 작업을 직접 처리하지 않고,
덜 중요한 스레드나 OS 에게 맡깁니다.
- 맡겨진 블로킹 작업이 완료되면,
후속 작업을 이어서 실행합니다.

이러한 구조 덕분에 Netty 는 스레드를 블로킹시키지 않고도
많은 요청을 동시에 처리할 수 있습니다.

이번 시간의 핵심 요약

- **중요한 스레드**는 대기하지 않는다.
- **블로킹 작업**은 OS 또는 별도의 **백그라운드 스레드**에게 위임한다.
- **후속 작업이 있다면 이어서 처리**, 없다면 종료.
- 이렇게 해서 전체 시스템의 응답성을 높인다.

스레드가 하나뿐이라면 비동기 처리는 불가능합니다.
대기 중인 스레드가 일을 계속할 수는 없으니까요.
그래서 항상 '대기를 담당할 다른 주체'가 필요합니다.

실제 비동기 동작 방식들

예시	비동기 처리 방식
setTimeout() in JavaScript	브라우저의 백그라운드 스레드가 대기
CompletableFuture in Java	지정한 별도 스레드에서 대기
Spring MVC + CompletableFuture	응답 스레드가 다른 스레드로 교체될 수 있음
Netty (WebFlux 기반)	OS의 I/O 다중화 (예: KQueue, epoll 등)가 대기
WebFlux 의 Flux/Mono	내부적으로 Scheduler의 스레드가 대기

비동기 처리의 예시 흐름

사용자 요청



중요한 스레드가 이벤트 등록만 하고 반환



블로킹 작업은 백그라운드 스레드가 처리



작업 완료 후 필요 시 후속 로직 실행



결과를 사용자에게 응답

이렇게 하면 중요한 스레드는 다른 요청을 계속 처리할 수 있게 되어 서버 전체의 효율성이 크게 향상됩니다.

다음 시간부터는 Flux, Mono 에서 자주 등장하는 **함수형 프로그래밍 개념들**을
먼저 살펴보겠습니다..

1.4 Reactive Stream – 함수형 프로그래밍이란?

이번 시간에는 **Flux** 와 **Mono** 가 사용하는 함수형 프로그래밍 개념에 대해 알아보겠습니다.

자바에서 우리가 일반적으로 작성하는 코드는 대입과 변수 재사용이 반복되는 구조로 작성되곤 하죠.

예를 들어, 어떤 리스트에 연산을 적용하고 조건을 걸고 다시 결과를 저장하는 이런 방식은...

- 가독성도 떨어지고,
- 유지보수성도 좋지 않습니다.

이런 문제를 해결하고 코드를 더 선언적으로, 간결하게 표현하기 위한 방식이 바로 **함수형 프로그래밍**입니다.

- 값을 두 배로 만들고
- 4 의 배수만 남기는 과정을 반복문으로 구현하고
- 중간 변수와 결과 리스트를 계속 사용

이런 스타일은 **한 눈에 로직이 잘 안 들어오고**, 수정할 때도 **부분 부분을 손대야** 하죠. 이제 이 로직을 함수형으로 표현해봅시다.

- 데이터를 변환하는 건 `map`
- 특정 조건을 통과시키는 건 `filter`
- 각각의 처리를 함수로 전달

```
list.stream()
    .map(n -> n * 2)
    .filter(n -> n % 4 == 0)
    .forEach(System.out::println);
```

- `map` → 각 요소를 *2

- filter → 4 의 배수만 남김
- forEach → 결과 출력

함수형 프로그래밍의 핵심은 '함수를 객체로 다루는 것'

함수를 변수에 담고, 다른 함수의 인자로 넘길 수 있다는 것이죠.

Java 에서 함수도 객체다?

Java 8 부터는 Function, Consumer, Supplier 등의 **함수형 인터페이스**를 통해 함수를 객체처럼 취급하고 전달할 수 있게 되었습니다.

함수형 프로그래밍 → 리액티브 스트림

이제 WebFlux 에서 사용하는 **Flux** 로 넘어가보겠습니다.

Flux 는 **자바 스트림과 매우 유사한 오퍼레이터(map, filter 등)**를 사용하지만, 중요한 차이가 하나 있습니다.

Flux 는 함수 덩어리를 가지고 있을 뿐, 바로 실행되지 않습니다.

예를 들어:

```
Flux<Integer> flux = Flux.range(1, 10)
    .map(n -> n * 2)
    .filter(n -> n % 4 == 0);
```

이렇게 작성해도 실제로는 아무 일도 벌어지지 않습니다.

왜냐하면 이건 **함수들의 체인을 정의한 것일 뿐, 실행 명령이 떨어지지 않았기 때문**입니다.

실행하려면 subscribe()를 호출해야 합니다.

Flux 는 구독(Subscribe)을 통해 실행이 시작됩니다.

```
flux.subscribe(System.out::println);
```

이제야 각 데이터가 map → filter 를 거쳐 forEach 처럼 소비되며 방출됩니다.

이 구조는 리액티브 스트림의 **Subscriber - Publisher** 패턴입니다.

함수형 스타일 정리

연산자	역할
map	값을 변환
filter	조건 필터링
flatMap	비동기 or 다중 값 처리
forEach (Stream) / subscribe (Flux)	실행 트리거 (최종 연산)

다시 강조! Flux 는 '함수 덩어리'

- Flux 는 데이터가 아니라, 아직 실행되지 않은 함수들의 체인입니다.
- 반드시 subscribe()가 호출되어야만 실행됩니다.
- WebFlux 에서는 이 subscribe()를 내부에서 자동으로 호출하여 실행합니다.

컨트롤러에서 Flux 를 그냥 반환했을 때도 결과가 나오는 이유는

Spring WebFlux 가 내부적으로 **Netty** 의 이벤트 루프 스레드에서 구독을 수행하기 때문입니다.

```
public Flux<Integer> example() {  
    return Flux.range(1, 9)  
        .map(n -> n * 2)  
        .filter(n -> n % 4 == 0);  
}
```

}

- 실행되지 않은 Flux
- Spring 이 자동 구독
- Netty 이벤트 루프가 결과 전송
- 작업 완료 후 Netty 로 복귀

이번 시간의 핵심 요약

1. **함수형 프로그래밍**은 함수를 객체처럼 다루며, 선언적 코드 스타일을 가능하게 합니다.
2. Flux, Mono 도 자바 스트림처럼 map, filter 등 오퍼레이터로 조합됩니다.
3. 하지만 Flux 는 ****구독(subscribe)****이 있어야 실행이 시작됩니다.
4. WebFlux 는 내부적으로 이 구독을 대신 해주기 때문에 컨트롤러에서 바로 사용 가능합니다.

이제 리액티브 스트림이 어떻게 **실제로 실행**되는지,
즉 **Publisher - Subscriber 패턴의 동작 방식**을 더 자세히 살펴보겠습니다.

1.5 WebFlux 의 Subscriber – Publisher 패턴

이번 시간에는 리액티브 프로그래밍의 핵심 패턴인 **Subscriber - Publisher** 패턴에 대해 살펴보겠습니다.

기본 개념

용어	의미
Publisher	데이터를 제공하는 주체
Subscriber	데이터를 구독하고 처리하는 소비자
subscribe()	구독 요청을 시작함으로써 데이터 흐름을 시작시키는 트리거
publish	데이터를 실제로 발행하는 동작

구독자가 구독을 요청해야지만 데이터가 발행됩니다.
이 구조는 **Flux** 나 **Mono** 에서도 동일하게 적용됩니다.

Flux 의 동작: 코드로 보는 예시

```
Flux<Integer> flux = Flux.range(1, 5)
    .map(i -> i * 2)
```

```
flux.subscribe(); // 구독이 있어야 실행됨!
```

- Flux.range()는 Publisher 역할
- subscribe()가 호출되어야 데이터 발행 시작

Flux 나 Mono 는 '데이터가 들어 있는 객체'가 아니라, 실행 전 상태의 함수 체인입니다.

비동기 처리를 위한 도구: Scheduler

단순 구독으로는 블로킹 회피가 안 됩니다.

Subscriber - Publisher 패턴 자체만으로는 블로킹을 회피할 수 없습니다.

Flux 와 Mono 는 내부적으로 **Scheduler** 라는 개념을 통해 **별도의 스레드를 활용**할 수 있도록 지원합니다. 이 추가 스레드를 활용하면 비동기 처리를 할 수 있습니다.

사용 예시

```
Flux.range(1, 5)
    .map(i -> i * 2)
    .subscribeOn(Schedulers.boundedElastic()) // 데이터 생성 로직을 별도
스레드에서 실행
    .doOnNext(System.out::println)
    .subscribe();
```

```
Flux.range(1, 5)
    .map(i -> i * 2)
    .publishOn(Schedulers.parallel()) // 이후 연산부터 병렬 스레드에서 실행
    .doOnNext(System.out::println)
    .subscribe();
```

연산자	역할
subscribeOn	데이터 생성(Publisher 측) 스레드를 지정

연산자	역할
publishOn	데이터 처리(Subscriber 측) 스레드를 지정

이 스케줄러가 제공하는 스레드는 **Netty의 이벤트 루프와 무관하게** 동작합니다. 따라서 여기서 블로킹이 발생해도 Netty에는 영향을 주지 않습니다.

이번 시간의 핵심 요약

1. Flux, Mono에는 실제 데이터가 들어있지 않고, subscribe()가 실행 트리거다.
2. 병렬 처리나 블로킹 회피를 위해서는 반드시 Scheduler를 통해 별도 스레드를 지정해야 한다.

콜드 시퀀스 vs 핫 시퀀스

구분	특징
콜드 시퀀스 (Cold Sequence)	구독자가 있어야 발행 시작 (Flux, Mono 대부분)
핫 시퀀스 (Hot Sequence)	구독자와 무관하게 발행 계속됨 (ex: publish())

일반적인 WebFlux 로직에서는 **콜드 시퀀스**를 사용합니다.

핫 시퀀스는 실시간 방송 스트림이 등에 적합하며, 백프레시 제어도 필요합니다.

WebFlux와 Subscriber - Publisher 패턴

Spring WebFlux는 이 패턴을 기반으로 동작합니다.

- 컨트롤러에서 Flux나 Mono를 반환하면
- WebFlux 내부에서 이를 구독하고
- Netty를 통해 클라이언트에게 응답을 전송합니다.

즉, 우리가 직접 `subscribe()`를 호출하지 않아도 실행되는 이유는,
WebFlux 내부에서 Netty의 이벤트 루프 스레드가 구독을 자동 수행하기
때문입니다.

다음 시간에는 Netty와 이벤트 루프(Event Loop)가 어떻게 작동하는지 자세히
알아보겠습니다.

1.6 Netty와 이벤트 루프 패턴

이전 시간까지 **Subscriber - Publisher** 패턴을 통해 리액티브 스트림의 동작
원리를 이해했다면,
이번 시간에는 리액티브 스트림이 **어떻게 실제로 실행되는지**,
즉 **Netty와 이벤트 루프 구조**를 통해 살펴보겠습니다.

왜 '이벤트 루프 패턴'이라고 할까요?

Tomcat, Netty, NIO 서버 등 비동기 웹서버는 공통적으로 이 이벤트 루프 패턴을 사용합니다.

- Tomcat 도 **NIO** 기반으로 설정하면 내부적으로 이벤트 루프를 사용합니다.
- Netty 는 대표적인 **이벤트 루프 기반 비동기 웹 서버**입니다.

이벤트 루프 패턴은 널리 쓰이는 비동기 처리 구조이며, WebFlux 도 Netty 의 이벤트 루프 위에서 동작합니다.

이벤트 루프 구조: 간단히 정리

구성 요소	역할
Selector	OS 가 감지한 이벤트를 확인
이벤트 루프 스레드	루프를 돌며 이벤트를 감지하고 처리
이벤트	네트워크 요청 수락, 요청 읽기, 응답 쓰기 등
Attachment	이벤트에 등록된 '처리 함수' (콜백 함수)

기본 흐름

1. OS 가 이벤트를 쌓아둡니다 (예: 클라이언트 요청 수락됨)
2. 이벤트 루프 스레드는 **selector** 를 통해 이벤트를 감지
3. 이벤트에 등록된 **callback 함수**를 실행
4. 다시 루프를 돌며 다음 이벤트를 기다림

스레드는 절대 대기하지 않고, OS 가 이벤트를 준비할 때까지는 놀고 있음
→ **완전한 논블로킹 구조**를 구현할 수 있는 비결입니다.

간단한 이벤트 루프 코드 구조 (개념 예시)

```
while (true) {
    selector.select(); // OS 에게 이벤트 있는지 물어봄
    Set<SelectionKey> keys = selector.selectedKeys();

    for (SelectionKey key : keys) {
        if (key.isAcceptable()) {
            // 요청 수락 -> 읽기 이벤트 등록
        } else if (key.isReadable()) {
            // 요청 읽기 -> 애플리케이션 로직 처리 + 응답 쓰기
        } else if (key.isWritable()) {
            // 응답 쓰기 완료
        }
    }
}
```

각 이벤트의 역할 & 후속 처리

이벤트	처리 내용
요청 수락 (Accept)	연결을 수락하고, 읽기 이벤트 등록
요청 읽기 (Read)	요청 내용을 읽고 → 사용자 코드 실행 → 응답 준비
응답 쓰기 (Write)	응답 데이터를 네트워크에 씀. OS 버퍼가 가득 차면 write 이벤트 등록

응답은 channel.write()를 통해 비동기적으로 처리됩니다.
다만, 버퍼가 가득 찼다면 나중에 다시 쓰도록 write 이벤트를 등록하게 됩니다.

이벤트 루프 스레드의 특징

- 반복 루프를 돌면서 이벤트가 있는지 확인
- 있다면 해당 이벤트에 달린 함수를 실행 (Callback)
- 스레드는 대기하지 않음, OS 가 이벤트 준비할 때까지 루프만 반복

모든 처리는 대기는 OS 에 위임하고

애플리케이션 로직 실행만 스레드가 맡습니다.

이번 시간의 핵심 요약

1. 이벤트 루프는 비동기 웹 서버의 핵심 처리 방식이다.
 2. Netty 는 이 구조 위에서 모든 요청을 논블로킹하게 처리한다.
 3. 이벤트에는 **콜백 함수(함수 객체)**가 등록되어 있으며, 이벤트가 발생하면 이를 실행한다.
 4. 이 구조 덕분에 스레드는 대기 없이 필요한 작업만 짧게 실행하고 바로 반환할 수 있다.
-

이제 무엇이 남았을까요?

- 우리는 이제 WebFlux 가 어떤 철학을 가지고 있고 어떻게 구현되는지,
 - Netty 가 어떻게 이벤트 루프 기반으로 동작하는지,
- 모두 이해했습니다.

그렇다면 다음으로 궁금한 것은?

WebFlux 와 Netty 가 어떻게 통합되는가입니다.

- WebFlux 는 우리가 컨트롤러에서 반환한 Flux 를 어디서 구독하고,

- 그 데이터를 어떻게 **Netty** 를 통해 사용자에게 응답으로 보내는지 직접 내부 구조를 따라가며 살펴보겠습니다.

다음 시간에는 **WebFlux** 와 **Netty** 의 통합 구조를 자세히 알아보겠습니다.

1.7 WebFlux 는 Netty 와 어떻게 통합 될까?

이제까지 우리는 WebFlux 가 Flux, Mono 를 통해 리액티브하게 데이터를 처리하고,
Netty 는 이벤트 루프를 통해 비동기적으로 네트워크 I/O 를 처리한다는 것을 배웠습니다.

그렇다면 궁금한 점이 하나 생기죠?

"WebFlux 는 어떻게 Netty 와 통합되어 데이터를 사용자에게 응답하는가?"

기본적인 요청 처리 흐름

가장 단순한 경우부터 살펴보겠습니다.
우리가 컨트롤러에서 Flux 만 반환하는 경우입니다.

기본 흐름 요약:

1. Netty 가 클라이언트 요청을 수락
2. 요청 읽기 이벤트 발생
3. WebFlux 가 이를 감지하고 Flux 를 구독
4. 로직 처리 후, 응답을 Netty 에 전달

이 경우에는 특별한 스레드 이동 없이, Netty 의 이벤트 루프 스레드가 처음부터 끝까지 흐름을 주도합니다.

Scheduler 를 사용한 경우: 응답을 누가 써야 할까?

우리가 subscribeOn 또는 publishOn 으로 스레드를 바꾸는 경우에는 상황이 달라집니다.

- Flux의 데이터를 별도 스레드에서 생성하거나 처리하게 되면,
- 그 스레드가 응답까지 직접 쓰려고 시도할 수 있습니다.

그런데 이건 문제가 됩니다.

왜 문제가 될까요?

- Netty는 응답을 ByteBuffer라는 구조에 기록하고,
- 이 버퍼는 스레드 안전하지 않습니다.
- 여러 스레드가 동시에 접근하면 응답 데이터가 뒤섞이거나 손상될 수 있습니다.

예시:

스레드 1이 "안녕하세요"를 쓰고 싶은데 "안녕하" 까지 쓴 순간,
스레드 2가 ByteBuffer에 접근해서 "반갑습니다!"를 쓰면
"안녕하반갑습니다!" 같이 엉망으로 응답 될 수 있습니다.

해결 방법: 다시 이벤트 루프 스레드로 위임

Netty는 이런 문제를 방지하기 위해 다음과 같은 처리를 합니다:

응답 쓰기 작업이 이벤트 루프 스레드가 아닐 경우 응답을 바로 쓰지 않고
이벤트 루프 스레드의 작업 큐에 등록합니다.

```
if (!executor.inEventLoop()) {  
  
    executor.execute(() -> 실제 응답 쓰기);  
}
```

이 처리가 들어있는 대표적인 클래스는
AbstractChannelHandlerContext의 write() 메소드입니다.

이벤트 루프 스레드만이 ByteBuffer를 안전하게 사용할 수 있기 때문에, 반드시
이 스레드에서 응답을 마무리해야 합니다.

외부 API 호출(WebClient)도 Netty 를 통해서?

WebClient 를 사용해서 외부 API 를 호출하는 경우도
Netty 기반의 비동기 처리 방식이 그대로 유지됩니다.

동작 흐름

1. WebClient 가 외부 API 요청을 보냄
2. 해당 요청의 대기는 OS 가 처리 (I/O 대기)
3. 응답이 도착하면 → **이벤트 루프 스레드는 등록된 콜백을 실행**
4. 결과 데이터는 다시 Flux 형태로 흘러감

WebClient 를 쓰면 **블로킹 없이 외부 API 요청이 가능해지고**,
Netty 의 이벤트 루프 스레드는 그동안 **다른 요청을 처리할 수** 있습니다.

RestTemplate 과 WebClient, 뭐가 다를까?

RestTemplate + Scheduler → 블로킹 대기를 별도 스레드에 위임
WebClient → 애초에 논블로킹 I/O 로 설계

항목	RestTemplate	WebClient
방식	블로킹	논블로킹
스레드 필요	별도 스레드 필요 (Scheduler)	없음 (OS 대기 + Netty)
성능	스레드 자원 소비 많음	자원 효율적

WebClient 는 블로킹 자체를 회피하기 때문에,
불필요한 스레드 낭비 없이 고성능 처리가 가능합니다.

이번 시간의 핵심 요약

1. **WebFlux** 는 요청을 받은 후 직접 **Flux** 를 구독하여 데이터를 방출합니다.
 2. 응답은 반드시 **Netty** 의 이벤트 루프 스레드에서 최종적으로 처리되어야 합니다.
 3. **WebClient** 를 사용하면 외부 API 호출까지도 **완전한 논블로킹 구조**로 처리 가능합니다.
 4. **Scheduler** 로 모든 블로킹 코드를 처리할 수는 있지만, **효율 측면에서 한계가 있으므로 WebClient 사용이 권장**됩니다.
-

이제 정말 마지막입니다.

- 스레드가 막히지 않도록 로직을 작성하는 방법
- 반드시 블로킹을 써야 할 경우 어떻게 격리할 수 있을지

다음시간에 이 부분을 정리하고, **WebFlux** 개념 강의를 마무리하겠습니다.

1.8 가장 중요

– WebFlux에서는 블로킹 처리를 어떻게 해야 할까?

WebFlux에서 블로킹은 어떻게 처리해야 할까?

강의 초반에 말씀드렸던 것처럼,

WebFlux는 외부 API 호출, DB 조회 등 블로킹 작업을 효율적으로 처리할 수 있는 구조라는 점이 큰 장점입니다.

이제 이 주제에 대해 본격적으로 정리해보겠습니다.

스케줄러로 블로킹을 피한 코드, 진짜 논블로킹일까?

우리는 이전에 **Scheduler**를 사용해 이벤트 루프 스레드를 블로킹하지 않도록 처리했습니다.

```
flux.subscribeOn(Schedulers.boundedElastic()) // 별도 스레드
```

이 방식은 이벤트 루프 스레드는 블로킹되지 않았기 때문에 시스템 전체에는 문제가 발생하지 않습니다.

하지만 진정한 의미에서 논블로킹이라고 보기는 어렵습니다.

왜냐하면 스레드 하나는 여전히 블로킹 상태이기 때문이죠.

리액티브 시스템의 핵심 철학: 스레드 변경 최소화

Netty가 비동기 + 논블로킹 방식으로 동작하는 이유는

- 컨텍스트 스위칭 비용을 줄이고

- 스레드 사용량을 최소화하여 성능을 높이기 위해서입니다.

그런데 블로킹마다 매번 스레드를 분리하면?

오히려 톰캣보다 성능이 떨어질 수 있습니다.

비교 항목	Tomcat	WebFlux (Scheduler 방식)
스레드 모델	요청당 1 스레드	요청 중간에 스레드 분리 발생
컨텍스트 스위칭	거의 없음	매번 발생
블로킹 처리	허용, 자연스러움	부자연스러움, 성능 저하

그렇다면 스레드 분리 없이, 블로킹 없이 외부 작업을 처리하려면 어떻게 해야 할까요?

OS에게 대기 맡기고, 이벤트로 처리하는 방식이 필요합니다.

이를 위해 아래와 같은 라이브러리 사용이 필요합니다.

용도	라이브러리	설명
외부 API 호출	WebClient	Netty 기반 비동기 HTTP 클라이언트
DB 조회	R2DBC	Reactive Relational DB Connectivity
Redis 캐싱	Reactive Redis	리액티브 Redis 클라이언트

이들을 사용하면

- Netty의 이벤트 루프는 절대 블로킹되지 않으며,
- OS의 이벤트 큐를 통해 완전히 논블로킹하게 처리됩니다.

WebClient는 어떻게 논블로킹일까?

예시 코드

```
return webClient.get()
    .uri("https://api.example.com/data")
    .retrieve()
    .bodyToFlux(String.class);
```

이 코드는 별도의 스케줄러를 할당하지 않았음에도 블로킹이 일어나지 않습니다.

- 이 Flux 는 일반적인 Flux.just(...) 같은 구현체가 아닌 FluxReceive 라는 **Netty 기반 Flux** 구현체입니다.
- subscribe()가 호출되더라도, **Netty 이벤트 루프 스레드는 대기하지 않습니다.**
- 응답이 준비되면 Netty 가 **콜백 함수**를 실행하여 데이터를 전달합니다.

즉, 스레드를 추가로 할당하지 않고도 완전한 비동기 처리가 가능한 것입니다.

그럼 Scheduler 는 언제 써야 하나요?

Scheduler 는 "어쩔 수 없이 블로킹이 발생하는 상황" 혹은 "병렬 처리를 명확히 의도할 때" 사용해야 합니다.

사용 예시:

- Thread.sleep() 등 블로킹 함수 사용
- JDBC, JPA 같은 **블로킹 기반 DB 클라이언트 사용**
- 외부 라이브러리가 비동기 처리를 지원하지 않을 때
- 여러 작업을 병렬로 실행하고 싶을 때

작업 유형	처리 방식
중요한 트랜잭션 작업 (회원가입, 로그인 등)	JPA + Scheduler (스레드 분리)
데이터 수집, 채팅 저장 등	MQ 사용 (완전 비동기 저장)
가벼운 조회, 중요도 낮은 데이터	R2DBC 사용

WebFlux 블로킹 처리 원칙 정리

1. **가능하면 R2DBC, WebClient, Reactive Redis 등 리액티브 라이브러리를 사용하자**
→ Netty 기반 완전한 논블로킹 구조
2. **불가피한 블로킹, 병렬 작업은 Scheduler 로 스레드를 분리하자**
→ boundedElastic, parallel 등 적절히 활용
3. **불필요한 스레드 전환은 피해야 한다**
→ 리액티브 프로그래밍의 본질은 '스레드 변경 최소화'

여기까지 오셨다면

- WebFlux 의 철학과 구조
- 이벤트 루프 기반 처리 방식
- Flux, Mono 의 동작
- Scheduler 와 스레드 처리 전략
- 그리고 진짜 논블로킹을 위한 리액티브 라이브러리 활용법

이 모든 것을 이해하셨습니다.

이제 개념 정리는 마무리되었습니다.

다음 장부터는 **2장 실습 파트**로 들어가,

직접 WebFlux 를 사용해보며 코드를 통해 개념을 체득해보겠습니다.

2 장 WebFlux, 어떻게 구현할까?

2.1 WebFlux 의 기본적인 사용법, Flux 와 Mono

이번 장의 목표

- Flux 와 Mono 의 차이점 이해하기
- 언제 어떤 걸 써야 할지 감 잡기
- 기본적인 예제 코드로 동작 방식 체험하기

Flux 와 Mono 란?

WebFlux 는 **리액티브 스트림의 구현체**인 Flux 와 Mono 를 주로 사용합니다. 이 둘은 모두 **Publisher** 의 구현체이며, 데이터를 구독하면 그때부터 발행이 시작됩니다.

구분	설명
Flux<T>	0 개 이상의 데이터를 발행
Mono<T>	0 또는 1 개의 데이터만 발행

Flux = List(Stream), Mono = Optional

이런 감각으로 접근하면 이해가 쉽습니다.

저는 처음에는 Mono 가 왜 필요한지 이해하기 어려웠습니다.

이론상 Flux 가 0 개에서 1 개의 데이터도 처리 가능하기 때문에 Mono 를 완전히 대체할 수 있기 때문입니다.

- 실제 로직에서 하나의 값만 다룰 때는 Mono 가 훨씬 직관적입니다.

- 예: 로그인 결과, 단건 조회 결과, 단순한 처리 응답 등

```
Mono<User> userMono = userService.findById(1L);
```

- Flux<User>로 작성하면 데이터 흐름이 오히려 과도하게 느껴질 수 있습니다.
- Mono<User>는 “하나 나올 거야”라고 **개발자의 의도를 명확하게 표현**할 수 있습니다.

Flux 를 잘 이해하면 **Mono** 는 자연스럽게 따라옵니다.

기본 코드 예시

```
@GetMapping("/flux")
public Flux<String> getFlux() {
    return Flux.just("Apple", "Banana", "Cherry");
}
```

```
@GetMapping("/mono")
public Mono<String> getMono() {
    return Mono.just("Only One");
}
```

- /flux 는 3 개의 데이터를 순차적으로 응답
 - /mono 는 1 개의 데이터만 응답
-

List 를 반환했는데 왜 잘 작동할까?

```
@GetMapping("/list")
public List<String> getList() {
    return List.of("a", "b", "c");
}
```

```
}
```

사실 WebFlux에서는 동기적인 값을 반환하면 내부적으로 자동으로 감싸줍니다.

- List를 반환하면 → Mono.just(List)로 자동 변환
- String, int, Map도 모두 마찬가지

```
// 내부적으로 이렇게 처리됨
```

```
return Mono.just(List.of("a", "b", "c"));
```

그래서 우리가 아무 생각 없이 List를 반환해도 WebFlux는 문제 없이 응답을 만들어냅니다.

이건 **WebFlux**의 편의 기능이자 유연함의 장점입니다.

그러나 명확한 의미 표현을 위해선 직접 Mono/Flux 사용을 권장

- 단건 데이터 → Mono<T>
- 컬렉션/스트리밍 → Flux<T>
- 처리 결과 없음 → Mono<Void>

```
@PostMapping("/save")
```

```
public Mono<Void> save(@RequestBody Item item) {
```

```
    return itemService.save(item)
```

```
        .then(); // 리턴값 없이 완료만 알림
```

```
}
```

이번 시간 핵심 정리

1. Flux는 0개 이상의 데이터를, Mono는 0~1개의 데이터를 발행합니다.
2. 단건 로직은 Mono를 사용하는 것이 더 명확하고 가독성이 좋습니다.

3. 동기 객체를 반환하면 WebFlux 가 자동으로 **Mono.just(...)**로 감싸줍니다.
4. 직접 **Flux/Mono** 를 사용하는 것이 리액티브 흐름을 명확하게 표현하는 방법입니다.

우리 map이나 filter 그리고 just, fromIterable이런 요소들을 지나치듯이 다뤘었는데 다음시간에는 이런 Operator들에 대해서 조금 더 자세하게 알아보겠습니다.

2.2 Operator 에 대하여

Operator 란?

앞서 개념 시간에 말씀드렸듯이,
Operator(연산자)는 데이터 흐름을 가공하고 제어하는 리액티브 프로그래밍의
핵심 구성 요소입니다.

모든 Operator 를 알아야 하나요?

그렇지 않습니다.

자주 쓰는 핵심 Operator 몇 개만 알아도 대부분의 코드 작성이 가능합니다.
기본적인것만 알고 있으면 대부분은 기본적인 Operator 을 조합해서 구현이
간으합니다.

공식 문서: <https://docs.spring.vmware.com/reactor-core/docs/3.5.21/docs/#which-operator>

궁금하면 한 번쯤 읽어보셔도 좋지만, 실전에서 부딪혀가며 익히는 게 훨씬
빠릅니다.

리액티브 흐름 구성하기

리액티브 코드는 보통 아래와 같은 흐름으로 구성됩니다.

우선 Flux 나 Mono 를 시작해야겠죠?

그래서 Flux 와 Mono 를 생성하는 Operator 가 필요하구요

그 이후에는 Flux 와 Mono 의 데이터를 가공하는 Operator 가 필요합니다.

그 후에는 Subscribe 를 하면서 데이터가 방출되구요.

그래서 정리하자면

Flux 와 Mono 를 생성하는 Operator

-> 데이터를 가공하는 Operator

-> subscribe

순서로 흐름이 구성됩니다.

아래에서 다룰

Flux 와 Mono 를 생성하는 Operator (just, defer, create, fromIterable)

데이터 가공 Operator (map, filter, flatMap, collectList)

정도만 외워두시는걸 추천드립니다.

시작: Flux 또는 Mono 생성

Mono 시작 방식

1. 데이터로 시작하는 경우

- **Mono.just(value)**

이미 존재하는 값 하나를 Mono 로 감쌌

```
Mono<String> mono = Mono.just("hello");
```

2. 함수로 시작하는 경우

- **Mono.defer() -> Mono.just(...)**

```
Mono<LocalDateTime> mono = Mono.defer(() ->  
    Mono.just(LocalDateTime.now()));
```

- **Mono.fromCallable()** -> **someBlockingCall()**

함수를 Mono 로 래핑

```
Mono<String> mono = Mono.fromCallable() -> "result from function");
```

Flux 시작 방식

1. 데이터로 시작하는 경우

- **Flux.just(v1, v2, ...)**

여러 개의 값을 직접 입력해서 생성

```
Flux<String> flux = Flux.just("a", "b", "c");
```

- **Flux.fromIterable(list)**

리스트나 컬렉션을 Flux 로 변환

```
Flux<Integer> flux = Flux.fromIterable(List.of(1, 2, 3));
```

2. 함수로 시작하는 경우

- **Flux.create(...)**

상태 없이 또는 상태 기반으로 값을 하나씩 생성
이에 대해서는 다음 시간에 더 자세하게 다룹니다.

```
Flux<String> flux = Flux.create(sink -> sink.next("hi"));
```

- **Flux.defer(() -> Flux.just(...))**

```
Flux<String> flux = Flux.defer() ->  
Flux.just(UUID.randomUUID().toString());
```

요약 정리

종류	데이터 기반 생성	함수 기반 생성
Mono	<code>Mono.just(value)</code>	<code>Mono.defer(...)</code> , <code>Mono.fromCallable(...)</code>
Flux	<code>Flux.just(...)</code> , <code>Flux.fromIterable(...)</code>	<code>Flux.defer(...)</code> , <code>Flux.create(...)</code>

흐름 가공: 데이터 처리

리액티브 스트림에서는 생성한 데이터 흐름을 가공(변환, 필터링, 결합)하여 원하는 결과로 만들 수 있습니다.

가장 기본이자 필수적인 연산자들을 아래와 같이 정리했습니다.

`map`, `filter`, `flatMap`, `collectList` 입니다.

1. `map()`

- **역할:** 각각의 데이터를 변환 (동기식)
- **특징:** 한 값을 받아 한 값으로 변환 (1:1 매핑)
- **적합한 상황:** 문자열을 대문자로 변환, 숫자에 연산 적용 등

```
Flux.just(1, 2, 3)
```

```
.map(i -> i * 2) // 결과: 2, 4, 6
```

`map()` 안에서 예외가 발생하면 스트림이 종료되므로 예외 처리는 주의!

2. `filter()`

- **역할:** 조건에 맞는 값만 통과시킴
- **특징:** `true` 인 값만 남기고 나머지는 제거
- **적합한 상황:** 짝수만 필터링, 특정 문자열만 남기기 등

```
Flux.just(1, 2, 3, 4, 5)
    .filter(i -> i % 2 == 0) // 결과: 2, 4
```

아무것도 통과하지 않으면 결과는 빈 Flux 가 됩니다.

3. flatMap()

- **역할:** 각 데이터를 비동기 함수로 변환하고 내부의 Mono/Flux 를 평탄화(flatten)
- **특징:** map()은 $T \rightarrow R$, flatMap()은 $T \rightarrow \text{Publisher}<R>$ (즉, **Flux<Mono<R>>** → **Flux<R>** 로 바꿔줌)
- **적합한 상황:**
 - Mono(Flux)흐름 안에서 외부 API,DB 호출 등으로 Mono 가 중첩된 경우

```
Flux.just(1, 2, 3)
    .flatMap(id -> userService.findUserById(id)) // 결과: Flux<User>
```

flatMap()은 내부 순서를 보장하지 않습니다. (병렬 실행 가능성 있음)
순서 보장이 필요하면 flatMapSequential()사용이 필요합니다.

평탄화가 무슨 소리이고 순서 보장이 무슨소리인지는

다음 시간에 더 자세하게 다룹니다.

4. collectList()

- **역할:** $\text{Flux}<T> \rightarrow \text{Mono}<\text{List}<T>>$ 로 수집
- **특징:** Flux 의 모든 데이터를 모아서 하나의 리스트로 반환
- **적합한 상황:** 결과를 리스트로 한번에 반환하고 싶을 때

```
Flux.just("a", "b", "c")
    .collectList() // 결과: Mono<List<String>> → ["a", "b", "c"]
```

주의: collectList()는 모든 데이터가 **완료된 이후**에 결과를 방출합니다.

구독: 흐름 실행 시작

```
flux.subscribe(data -> System.out.println("데이터: " + data));
```

구독이 없으면 아무 일도 일어나지 않습니다.

리액티브 스트림은 구독 된 이후에 실행되기 때문입니다.

주의: 블로킹 요소가 들어가면 반드시 **Scheduler** 사용

```
Mono.just("start")
    .map(s -> {
        Thread.sleep(1000); // 블로킹
        return s.toUpperCase();
    })
    .subscribe(); // Netty 이벤트 루프 블로킹 발생!
```

이런 코드에서는 반드시 `subscribeOn(Schedulers.boundedElastic())`처럼 **스레드 분리**가 필요합니다.

에러 처리

에러는 어디서 발생하고 어디서 잡히는가?

```
Flux.range(1, 5)
    .map(i -> {
        if (i == 3) throw new RuntimeException("에러 발생!");
        return i;
    })
    .onErrorReturn(-1)
    .subscribe(System.out::println);
```

리액티브 스트림을 사용하다 보면 이런 의문을 가지게 됩니다.

“앵? 에러는 *map* 에서 발생했는데, 왜 다음 operator *onErrorReturn* 에서 잡히는 거예요?”

여기서 3 일 때 예외가 발생하지만, *onErrorReturn* 에서 **정상적으로 처리**됩니다.
예외가 바깥으로 터지지 않고 스트림 내부에서 잡힌다는 것이죠.

리액티브 스트림에서는

스트림 안에서 발생한 예외는 흐름 내부에서 자동으로 포착됩니다.

즉, *map()*, *filter()*, *flatMap()* 같은 연산자 내부에서 예외가 발생하더라도
그건 바깥으로 던져지는 게 아니라, **스트림의 에러 흐름으로 전환**됩니다.

물론 직접 ***try-catch*** 로 처리할 수도 있습니다.

특히, 특정 연산자 내부에서 **부분적으로 예외를 컨트롤**하고 싶을 때는 *try-catch* 도 유용합니다.

```
.map(i -> {  
    try {  
        if (i == 3) throw new RuntimeException("에러");  
        return i * 2;  
    } catch (Exception e) {  
        return -1;  
    }  
})
```

이 방식은 마치 *onErrorReturn()* 처럼 작동하지만,
스트림 흐름이 끊기지 않고 이어질 수 있다는 점에서 유용합니다.

이에 대한 내용은 에러처리 구현에서 더 자세하게 살펴봅니다.

디버깅용: *doOnNext()*, *doOnError()*, *doFinally()*

이들은 흐름을 변형하진 않지만 **디버깅, 로깅, 트래킹**에 유용한 유틸리티 연산자입니다. 사용 용도는 해석 그대로입니다.

doOnNext: 데이터가 방출될 때 마다 트리거

doOnError: 에러가 발생할 때 마다 트리거

doFinally: 스트림이 완료되면 트리거

Flux.just(1, 2, 3)

```
.doOnNext(i -> log.info("방출됨: " + i))  
.doOnError(e -> log.error("에러 발생", e))  
.doFinally(sig -> log.info("스트림 종료: " + sig))
```

핵심 요약

1. **Operator** 는 데이터를 변환, 필터링, 결합, 에러처리 등 다양한 흐름을 구성하는 도구입니다.
2. 핵심 **Operator** 몇 개만 익혀도 대부분의 구현은 해결됩니다.
3. **Flux/Mono** 시작 → 가공(map/filter/flatMap) → 구독 → 실행 이 흐름을 기억하세요.
4. 블로킹 연산은 **Scheduler** 로 격리해야 안정적인 비동기 처리가 가능합니다.

2.3 Operator FlatMap 의 특성

FlatMap 은 비동기 + 비동기가 중첩된 구조, 즉, `Mono<Mono<String>>` 과 같은 구조를 `Mono<String>` 으로 바꿔줍니다.

비동기 + 비동기?

비동기 작업 안에서 또 다른 비동기 작업을 호출하면 "언제 끝날지 모르는 작업"이 겹치는 구조가 만들어집니다.

예를 들어,

```
Mono<Mono<String>> mono = Mono.just("id")
    .map(id -> 외부 API 호출(id)); // 외부 API 가 Mono 를 반환
```

이렇게 되면 `Mono<Mono<String>>` 형태가 되죠.

그런데 비동기 작업 + 비동기 작업 = 결국 **하나의 비동기 작업** 아닐까요?

언제 끝날지 모르는 일 + 언제 끝날지 모르는 일이 합쳐졌어요
그러면 그냥 언제 끝날지 모르는 일 한 개로 봐도 괜찮다는 말입니다.

마치 수학에서:

무한대 + 무한대 = 무한대

이와 비슷하게:

`Mono<Mono<T>>` → `Mono<T>` 로 간주해도 무방하다는 의미입니다.

이처럼 비동기 안에 또 비동기가 있는 경우,
이중 구조를 **한 단계로 평탄화(flatten)** 해주는 연산자가 바로 flatMap 입니다.

```
Mono<String> mono = Mono.just("id")  
    .flatMap(id -> 외부 API 호출(id)); // Mono<String>
```

중첩된 흐름을 하나로 정리해주는 것이 flatMap 의 역할입니다.

-
- 비동기 흐름 안에서 또 다른 비동기를 호출하면 Mono<Mono<T>>처럼 중첩됨
 - 이걸 **단일 흐름 Mono<T>로 바꿔주는 연산자가 flatMap**
 - 외부 API 호출, DB 조회, 비동기 연산 등으로 인해 **Mono 가 중첩 될 때 사용하기 용이하다.**
-

리액티브 스트림에서 flatMap 은 **중첩된 비동기 작업을 처리하는 데** 매우 유용한 연산자입니다.

하지만 한 가지 중요한 특징이 있습니다.

flatMap 은 데이터가 입력된 순서를 보장하지 않습니다.

이는 flatMap 이 입력된 순서 대신 내부 요소들이 처리 완료되는 순서대로 데이터를 방출하기 때문입니다.

예시를 살펴볼까요?

```
Flux.just(1, 2, 3)  
    .subscribeOn(Schedulers.boundedElastic())  
    .flatMap(id -> callApi(id))
```

이 코드는 callApi(1), callApi(2), callApi(3)을 **동시에 병렬로 실행**합니다.
그리고 결과는 **작업이 완료된 순서대로 방출**됩니다.

예를 들어, 2 번 작업 → 3 번 작업 → 1 번 작업 순으로 끝났다면
→ 결과도 **2, 3, 1** 순으로 전달됩니다.

요약하면

- flatMap 은 내부 호출을 **동시에 실행**합니다.
- **먼저 끝난 작업부터 방출**되기 때문에
- 결과는 입력 순서와 **달라질 수 있습니다.**
- 즉, 데이터가 **도착하는 즉시 처리**되며, **입력 순서를 고려하지 않습니다.**

flatMap 과 오해

flatMap이 내부적으로 비동기적으로 동작한다고 써있어서 아 flatMap을 쓰면
알아서 비동기적으로 블로킹이 회피 되겠구나라고 이해되는 경우가 있는 것
같습니다.

그런데 flatMap을 쓴다고 그냥 블로킹이 회피 될까요?

그럴 수가 없습니다.

저희 1장에서 비동기 프로그래밍에 대해 공부했을때를 잠시 기억해봅시다
비동기란 뭐예요? 특정 작업을 처리하고 있을 때 완료될 때 까지 기다리지 않고
다른 일을 하러 가버리는거잖아요

flatMap을 쓴다고 일하던 스레드가 도망갈까요?

아니죠. flatMap은 단지 흐름을 평탄화(flat)할 뿐, 스레드 이동이나 블로킹
회피와는 전혀 관련이 없습니다.

블로킹 회피를 하려면 스케줄러 할당이나 Netty의 지원이 필수입니다.

`flatMap`은 Mono<Mono<T>> 같은 중첩된 비동기 구조에서

내부 Publisher 가 완료되는 즉시 값을 꺼내

스트림 상에 하나의 Publisher 처럼 이어지도록 도와주는 연산자일 뿐입니다.

flatMap 이 비동기적으로 동작한다고 쓰여 있는 부분은

데이터의 입력 순서와 무관하게 데이터의 처리가 완료되는 순서대로 비동기적으로 값을 방출시키는 부분에 대한 설명이 적혀있는 것 입니다.

그러면 flatmap 에서 데이터 방출 순서를 지키고 싶으면 어떻게 해야 할까요?

순서가 중요한 상황에서는 flatMap 은 **예상치 못한 결과**를 만들어낼 수 있습니다.

예를 들어:

- 요청한 순서대로 응답을 받아야 하는 API 응답 처리
- 순서가 중요한 로그, 메시지 스트림, 파일 쓰기 작업 등

이런 상황에서는 flatMapSequential()을 사용해야 합니다.

flatMapSequential 이란?

flatMapSequential()은 다음과 같은 특징을 가집니다:

- 내부 비동기 작업은 flatMap 처럼 **동시에 실행됩니다.**
- 하지만 **출력은 입력된 순서대로 보장됩니다.**

즉, 작업은 병렬로 처리하면서도

결과는 1 → 2 → 3 순서대로 방출되게 만드는 연산자입니다.

```
Flux.just(1, 2, 3)
    .flatMapSequential(id -> callAsyncApi(id))
```

여러 개의 비동기 작업을 효율적으로 처리하면서도

정해진 순서를 유지해야 할 때 매우 유용한 연산자입니다.

다만 **정해진 순서를 보장하기 위해**

만약 1 번 데이터 처리가 아주 길어진다면 2,3 번 데이터의 방출도 같이 느려진다는 단점도 있습니다.

2.4 Sink 와 Context 에 대하여

sink 는 Flux.create 로 만들 수 있습니다.

sink.next 를 할 때 마다 Flux 의 요소들이 방출되구요

Sink 가 유용한 상황

1. 재귀 호출 구조

재귀 호출 구조는 연산자로 표현하기가 복잡합니다.

이건 나중에 직접 재귀 구조의 코딩을 하시다 보면 자연스레 필요가 느껴지실거예요.

2. 기존 코드와 통합

기존 코드와의 자연스러운 연결점

이 부분은 강의 시작 때 기존 코드 스타일을 어느정도 유지하면서 통합 가능한것을 살펴봤습니다.

3. 데이터 방출 타이밍을 쉽게 컨트롤 하고 싶을 때

sink 의 방출 타이밍을 보면 어때요?

우리가 next 하는 타이밍에 아주 명시적으로 방출됩니다.

이런 방출을 아주 쉽게 컨트롤할 수 있습니다.

Sink 와 Request 개수

- 리액티브 스트림은 기본적으로 구독자가 "얼마나 데이터를 받고 싶은지" 요청할 수 있는 구조입니다. 기본값은 무한대입니다.(request(n))

그런데 Sink 를 사용하면

- sink 는 우리가 직접 next() 호출로 방출하므로 요청 개수(request(n))를 설정하면 요청 개수 이상의 데이터가 next 로 방출 될 시 예외가 발생합니다. 따라서 sink 사용 시 request 개수 조작은 권장되지 않습니다.
-

Context 란?

리액티브한 흐름 안에서 값을 전달하는 안전한 방법입니다.

어디서든 접근 가능한 값이 필요한데 매번 매개변수로 넘기기 번거롭거나, 코드 흐름상 전달이 어려운 경우가 있습니다.

어디서든 접근이 필요한 값의 예시로는 사용자 ID, 인증 정보, 로깅용 트레이스 ID 등이 있을 수 있습니다.

이럴 때 쓰는 게 바로 리액티브 **Context** 입니다.

그런데, **ThreadLocal** 을 쓰면 안 될까요?

결론부터 말씀드리자면 **WebFlux** 에서는 **ThreadLocal** 사용이 매우 제한적입니다.

왜냐하면 WebFlux 는 **Netty** 기반의 비동기 처리 모델을 사용하기 때문입니다.

요청을 처리하는 동안 스레드가 수시로 바뀔 수 있고,

특히 우리가 자주 사용하는 subscribeOn, publishOn, Schedulers 같은 연산자를 사용하게 되면

스레드 변경은 거의 일상적으로 일어나게 됩니다.

그래서 Context 를 씁니다.

Context 는 리액터 내부에서 제공하는 **Key-Value 형태의 저장소**입니다.
스트림과 함께 **흐름을 따라 이동**하므로,
스레드 변경에도 안전하게 데이터 전달이 가능합니다.

Context 사용 예시

```
Mono.deferContextual(ctx -> {  
    String userId = ctx.get("userId");  
    return Mono.just("사용자: " + userId);  
})  
.contextWrite(Context.of("userId", "abc123"));
```

이렇게 하면 어디에서든 userId 에 접근할 수 있습니다.

JPA 트랜잭션은 그럼 못 쓰나요?

JPA 의 트랜잭션은 내부적으로 ThreadLocal 을 사용합니다.
그래서 일반적인 WebFlux 흐름에서는 **사용이 어렵습니다**.

다만 boundedElastic 으로 스레드를 분리하고 해당 스레드 안에서만

컨트롤 가능한 범위에서 JPA 트랜잭션의 지원을 받을 수 있기는
합니다.Schedulers.boundedElastic() 스레드 안에서는 ThreadLocal 이 유지됩니다

```
Mono.fromCallable() -> {  
    // JPA 트랜잭션 처리  
})  
.subscribeOn(Schedulers.boundedElastic());
```

단, 당연하지만 이 스레드 밖으로 트랜잭션 객체가 나가면 **깨집니다**.

Context + Sink

Sink 가 몇 번 방출됐는지 카운팅해보고 모두 방출됐다면 종료시켜보자

```
Flux.create(sink -> {
    sink.contextView(ctx -> {
        AtomicInteger counter = ctx.get("count");

        sink.next("data1");
        if (counter.incrementAndGet() == 0) {
            sink.complete();
        }

        sink.next("data2");
        if (counter.incrementAndGet() == 0) {
            sink.complete();
        }
    });
})
.contextWrite(Context.of("count", new AtomicInteger(2)));
```

2.5 Scheduler 에 대하여

Flux, Mono 에서 사용할 스레드를 관리하는 도구입니다.

- 기본적으로 Reactor 는 비동기 스트림을 통해 스레드 전환 없이 처리합니다.
- 하지만 블로킹 작업 또는 명확한 스레드 분리가 필요한 경우에는 Scheduler 를 사용하여 별도 스레드를 추가로 할당할 수 있습니다.

Scheduler 의 종류

대표적으로 boundedElastic과 parallel이 있습니다.

각 스케줄러 스레드의 가장 큰 차이점은 생성 방법과 생존 주기입니다.

1. boundedElastic

- 호출 시 필요한 만큼 스레드를 즉시 생성
- 일정 시간 미사용 시 스레드를 자동 제거
- 다음 호출 시 기존 스레드를 재사용
- 생성 가능한 최대 스레드 수는 상한선(bound)으로 제한됨

이름처럼 "유연하게(Elastic), 그러나 제한된 범위 내에서(bounded)" 스레드를 생성합니다.

적합한 작업:

- 블로킹 가능성이 높은 작업
- 예: 파일 IO, DB 접근, HTTP 요청 등

2. parallel

- 최초 호출 시, CPU 코어 수와 동일한 개수의 스레드 생성
- 삭제되지 않고 유지
- 스레드 수가 고정 → 블로킹에 적합하지 않음

적합한 작업:

- CPU 바운드 작업 (계산, 암호화, 데이터 처리)

그럼 어떤 걸 써야 하나요?

- 잘 모르겠으면 **boundedElastic** 만 써도 충분합니다.
- parallel 은 성능 최적화에 신경쓸 때만 사용하세요.

스케줄러 스레드는 어떻게 할당하나요?

리액티브 스트림에서 스레드를 바꾸는 연산자는 두 가지입니다:

1. subscribeOn(scheduler)

- 구독 시점부터 어느 스레드에서 작업을 시작할지 결정
- 예: DB 호출, 외부 API 처럼 시작부터 블로킹일 수 있는 작업

```
Mono.fromCallable(...)  
    .subscribeOn(Schedulers.boundedElastic());
```

2. publishOn(scheduler)

- 호출된 시점 이후의 연산은 **지정한 스레드에서 실행**
- 중간 흐름 중 **특정 연산만 스레드 분리**하고 싶을 때 사용합니다. 즉, `publishOn` 이 호출된 다음 작업 부터 스레드가 분리됩니다.

```

Mono.just("data")
    .map(data -> transform(data))
    .publishOn(Schedulers.parallel()) //이 코드 아래부터 스레드가 변경됩니다.
    .flatMap(this::callAnotherService);

```

Scheduler 사용의 효과

- 이전부터 계속 다뤘지만 스케줄러를 쓰면 **원하는 위치에 스레드를 할당**할 수 있습니다.
- 이는 Spring 의 `@Async`, `ExecutorService` 와 비슷한 효과입니다.

Reactor 는 기본적으로 "**필요할 때만 스레드 전환**"을 설계 철학으로 삼기 때문에 스케줄러 스레드를 남용 한다면 성능 저하를 유발할 수 있습니다. 그렇기 때문에 IO 라이브러리를 적절히 사용하여 블로킹 처리에는 스케줄러 대신 Netty 의 지원을 받는 것이 좋습니다.

정리

항목	설명
<code>boundedElastic</code>	블로킹 IO 에 적합 (유연하고 제한된 스레드 생성)
<code>parallel</code>	계산 중심 작업에 적합 (고정된 스레드 수)
<code>subscribeOn</code>	시작 스레드 지정
<code>publishOn</code>	이후 흐름의 스레드 전환
사용 시 주의	과도한 사용은 성능 저하 유발

이번 장 이후로는 모두 실습으로 진행되며
각 시간에 주석이 포함된 완성 코드를 첨부할 예정입니다.